

PlantUML > Расширенное использование > Командная строка >



Новинка! Отображайте диаграммы PlantUML прямо в GitHub с помощью нашего [официального расширения для браузера](#) – без сервера. Без токенов. Без отслеживания. Нулевые разрешения, кроме буфера обмена. – Попробуйте и поделитесь своим мнением!

[Добавить в Chrome](#)

[Добавить в Firefox](#)

📄 Командная строка

PlantUML можно запустить из командной строки. (См. раздел [«Запуск»](#) для получения информации о способах запуска PlantUML из различных других инструментов и рабочих процессов).

Самый простой способ запустить его:

```
java -jar plantuml.jar file1 file2 file3
```

Эта программа будет искать данные @startXYZ в файлах file1, file2 и file3. Для каждой диаграммы .png будет создан отдельный файл.

Для обработки всего каталога можно использовать:

```
java -jar plantuml.jar "c:/directory1" "c:/directory2"
```

Эта команда будет искать файлы в каталогах @startXYZ и .

```
@endXYZ .txt .tex .java .htm .html .c .h .cpp .apt .pu .puml .hpp .hh .md c:/directory1 c:/directory2
```

Образы Docker распространяются в виде [пакетов на Github](#) и на [Docker Hub](#).

```
docker run ghcr.io/plantuml/plantuml
```



📄 Новый интерфейс командной строки (бета-версия)

В настоящее время мы перерабатываем параметры командной строки PlantUML, чтобы они лучше соответствовали [стандартам командной строки GNU](#).

Это означает:

- Новые **опции** отличаются единообразным именованием и структурой.
- Поддержка устаревших **функций** будет продолжена в течение переходного периода, но их документация больше не будет вестись.

👉 Ваши отзывы на данном этапе очень ценны! Приглашаем вас протестировать продукт и сообщить нам, что вам кажется понятным (или непонятным).

Ниже приведён полный список опций, которые будут доступны в будущей бета-версии:

plantuml - generate diagrams from plain text

Usage:

```
java -jar plantuml.jar [options] [file|dir]...
java -jar plantuml.jar [options] --gui
```

Description:

Process PlantUML sources from files, directories (optionally recursive), or stdin (-pipe).

Wildcards (for files/dirs):

- * any characters except '/' and '\'
 - ? exactly one character except '/' and '\'
 - ** any characters across directories (recursive)

Tip: quote patterns to avoid shell expansion (e.g., "**/*.puml").

General:

```
-h, --help ..... Show this help
--help-more ..... Show extended help (advanced options)
--version ..... Show PlantUML and Java version
--author ..... Show information about PlantUML authors
--gui ..... Launch the graphical user interface
--dark-mode ..... Render diagrams in dark mode
-v, --verbose ..... Enable verbose logging
--duration ..... Print total processing time
--progress-bar ..... Show a textual progress bar
--splash-screen ..... Show splash screen with progress bar
--check-graphviz ..... Check Graphviz installation
--http-server[:<port>] ..... Start internal HTTP server for rendering (default port : 4242)
```

Input & preprocessing:

```
-p, --pipe ..... Read source from stdin, write result to stdout
-D<var>=<value>,
  --define <VAR>=<value> ..... Define a preprocessing variable (equivalent to '!define <var> <value>')
-I<file>,
  --include <file> ..... Include external file (as with '!include <file>')
-P<key>=<value>,
  --pragma <key>=<value> ..... Set pragma (equivalent to '!pragma <key> <value>')
-S<key>=<value>,
  --skinparam <key>=<value> .... Set skin parameter (equivalent to 'skinparam <key> <value>')
--theme <name> ..... Apply a theme
--config <file> ..... Specify configuration file
--charset <name> ..... Use a specific input charset
```

Execution control:

```
--check-syntax ..... Check diagram syntax without generating images
--stop-on-error ..... Stop at the first syntax error
--check-before-run ..... Pre-check syntax of all inputs and stop faster on error
--no-error-image ..... Do not generate error images for diagrams with syntax errors
--graphviz-timeout <seconds> Set Graphviz processing timeout (in seconds)
--threads <n|auto> ..... Use <n> threads for processing (auto = available processors)
```

Metadata & assets:

```
--extract-source ..... Extract embedded PlantUML source from PNG or SVG metadata
--disable-metadata ..... Do not include metadata in generated files
--skip-fresh ..... Skip PNG/SVG files that are already up-to-date (using metadata)
--sprite <4|8|16[z]> <file> .. Encode a sprite definition from an image file
--obfuscate ..... Obfuscate diagram texts for secure sharing
--encode-url ..... Generate an encoded PlantUML URL from a source file
```

```
--decode-url <string> ..... Decode a PlantUML encoded URL back to its source
--list-keywords ..... Print the list of PlantUML language keywords
--dot-path <path-to-dot-exe> Specify the path to the Graphviz 'dot' executable
--ftp-server ..... Start a local FTP server for diagram rendering (rarely used)
```

Output control:

```
--output-dir <dir> ..... Generate output files in the specified directory
--overwrite ..... Allow overwriting of read-only output files
--exclude <pattern> ..... Exclude input files matching the given pattern
```

Output format (choose one):

```
--format <name>, -f <name> ... Set the output format for generated diagrams
                                (e.g. png, svg, pdf, eps, latex, txt, utxt)
```

Available formats:

```
--eps ..... Generate images in EPS format
--html ..... Generate HTML files for class diagrams
--latex ..... Generate LaTeX/TikZ output
--latex-nopreamble ..... Generate LaTeX/TikZ output without preamble
--pdf ..... Generate PDF images
--png ..... Generate PNG images (default)
--scxml ..... Generate SCXML files for state diagrams
--svg ..... Generate SVG images
--txt ..... Generate ASCII art diagrams
--utxt ..... Generate ASCII art diagrams using Unicode characters
--vdx ..... Generate VDX files
--xmi ..... Generate XMI files for class diagrams
--preproc ..... Output preprocessor text of diagrams
```

Statistics:

```
--disable-stats ..... Disable statistics collection (default behavior)
--enable-stats ..... Enable statistics collection
--export-stats-html ..... Export collected statistics to an HTML report and exit
--export-stats ..... Export collected statistics to a text report and exit
--html-stats ..... Output general statistics in HTML format
--xml-stats ..... Output general statistics in XML format
--realtime-stats ..... Generate statistics in real time during processing
--loop-stats ..... Continuously print usage statistics during execution
```

Examples:

```
# Process all .puml recursively
java -jar plantuml.jar "**/*.puml"

# Check syntax only (CI)
java -jar plantuml.jar --check-syntax src/diagrams

# Read from stdin and write to stdout (SVG)
cat diagram.puml | java -jar plantuml.jar --svg -pipe > out.svg

# Encode a sprite from an image
java -jar plantuml.jar --sprite 16z myicon.png

# Use a define
java -jar plantuml.jar -DAUTHOR=John diagram.puml

# Change output directory
java -jar plantuml.jar --format svg --output-dir out diagrams/
```

Exit codes:

- 0 Success
- >0 Error (syntax error or processing failure)

See also:

java -jar plantuml.jar --help-more
Documentation: <https://plantuml.com>

Спасибо за вашу помощь и отзыв!

Подстановочные карты

Также можно использовать подстановочные знаки:

- Для одного символа используйте ?
- Для нуля или более символов используйте *
- Для нуля или более символов (включая / или \) используйте символ с плавающей запятой. **

Таким образом, для обработки любых .cpp файлов во всех каталогах, начинающихся с *фиктивного имени* :

```
java -jar plantuml.jar "dummy*/*.cpp"
```

А также для обработки любых .cpp файлов во всех каталогах, начинающихся с *dummy* , и их подкаталогах:

```
java -jar plantuml.jar "dummy*/**.cpp"
```

Исключенные файлы

Вы можете исключить некоторые файлы из процесса, используя следующую -x опцию:

```
java -jar plantuml.jar -x "**/common/**" -x "**/test/Test*" "dummy*/**/*.cpp"
```

Выходной каталог

С помощью ключа можно указать выходной каталог для всех изображений -o :

```
java -jar plantuml.jar -o "c:/outputPng" "c:/directory2"
```

При рекурсивном обходе нескольких каталогов существует небольшая разница в зависимости от того, указываете ли вы абсолютный или относительный путь к выходному каталогу:

- Указание абсолютного пути гарантирует, что все изображения будут выведены в одну конкретную директорию.
- Если указан относительный путь , то изображения помещаются в этот каталог относительно расположения входного **файла** , а не текущего каталога (примечание: это относится даже к случаю, когда путь начинается с символа '.' .). Когда Plantuml обрабатывает файлы из нескольких каталогов, соответствующая структура каталогов создается в вычисленном выходном каталоге.

См. раздел «Именованые файлы» в разделе «[Источники](#)» , где описано, как вычисляются имена выходных файлов при использовании нескольких диаграмм в одном входном файле.

Типы выходных файлов

Изображения для ваших диаграмм можно экспортировать в различных форматах. По умолчанию используется файл PNG, но можно выбрать другой формат, используя следующие расширения:

Имя параметра	Краткое название параметра	Формат вывода	Комментарий
--png	-f png	ПНГ	По умолчанию
--svg	-f svg	SVG	Векторная графика – используйте для веб-сайтов и документации (<i>подробнее: SVG</i>)
--eps	-f eps	EPS	Вывод в формате PostScript/EPS (<i>подробнее: EPS</i>)
--eps + постобработка	-f eps	EPS	Сохраняет текст EPS в текстовом формате – в зависимости от инструмента/версии (<i>см. EPS</i>).
--format pdf	-f pdf	PDF	Вывод в формате PDF (<i>подробнее: PDF</i>)
--format vdx	-f vdx	VDX	Документ Microsoft Visio (если поддерживается вашей сборкой PlantUML)
--format xmi	-f xmi	11-й	Обмен данными UML (если поддерживается) (<i>см. XMI</i>)
--format scxml	-f scxml	SCXML	Диаграмма состояний в формате XML (если поддерживается)
--format html	-f html	HTML	Экспериментальная/альфа-версия – не использовать в производственной среде без проверки поддержки.
--txt	-f txt	ATXT	ASCII-графика. (<i>Подробнее: ASCII-графика</i>)
--utxt	-f utxt	UTXT	ASCII-графика с использованием символов Юникода (<i>Подробнее: ASCII-графика</i>)
--latex	-f latex	ЛАТЕКС	Вывод в LaTeX/TikZ (<i>подробнее: LaTeX</i>)
--format latex + варианты	-f latex	ЛАТЕКС	пreamble Поведение зависит от версии – обратитесь к соответствующему разделу. --help-more
--format braille	-f braille	PNG (шрифт Брайля)	Изображение, напечатанное шрифтом Брайля (<i>может быть доступно в некоторых сборках; см. QA-4752</i>).

Пример:

```
java -jar plantuml.jar yourdiagram.txt --txt
```

🔗 📄 Файл конфигурации

Вы также можете указать файл конфигурации, который будет добавляться перед каждой диаграммой:

```
java -jar plantuml.jar -config "./config.cfg" dir1
```

🔗 📄 Метаданные

После всей предварительной обработки (включая и т. д.) PlantUML сохраняет исходный код диаграммы в сгенерированных метаданных PNG в виде [закодированного текста](#) .

- Если вы не хотите, чтобы PlantUML сохранял исходный код диаграммы в сгенерированных метаданных PNG, вы можете во время генерации использовать опцию `-nometadate` отключения этой функции (чтобы НЕ экспортировать метаданные в сгенерированные файлы PNG/SVG).
- Этот исходный код можно получить с помощью соответствующей `-metadata` опции. Это означает, что PNG-файл практически "редактируемый": его можно разместить на корпоративной вики, где нельзя устанавливать плагины, и кто-то в будущем сможет обновить диаграмму, получив метаданные, отредактировав и загрузив её заново. Кроме того, диаграмма является автономной.
- Напротив, эта `-checkmetadata` опция проверяет, совпадает ли исходный файл целевого PNG-файла с исходным, и если изменений нет, не регенерирует PNG-файл, тем самым экономя время обработки. Это позволяет запускать PlantUML для всей папки (или дерева файлов с этой `-recursive` опцией) инкрементально.

Звучит как волшебство! Нет, просто гениальная инженерная разработка :-)

Пример:

```
java -jar plantuml.jar -metadata diagram.png > diagram.puml
```

К сожалению, этот вариант работает только с локальными файлами. Он не работает с другими файлами -pipe, поэтому вы не сможете получить URL-адрес, например curl, и передать PNG-файл в PlantUML.

Однако [сервер](#) Plantuml обладает аналогичной функцией, позволяющей получать PNG-изображение по URL-адресу и извлекать из него метаданные.

🔗 Код выхода

При обнаружении ошибок в диаграммах команда возвращает код завершения с ошибкой (-1). Но даже если в некоторых диаграммах есть ошибки, генерируются **все диаграммы, что может занять много времени для крупных проектов.**

Вы можете использовать этот -failfast флаг, чтобы изменить поведение и остановить генерацию диаграмм, как только возникнет хотя бы одна ошибка. В этом случае некоторые диаграммы будут сгенерированы, а некоторые – нет.

Также есть -failfast2 флаг, который выполняет первую проверку. Если обнаруживается ошибка, диаграмма вообще не будет сгенерирована. В случае ошибки, -failfast2 программа работает даже быстрее, чем -failfast, что может быть полезно для крупных проектов.

🔗 Стандартный отчет [stdrpt]

С помощью -stdrpt опции (стандартный отчет) вы можете изменить формат вывода ошибок в ваших скриптах PlantUML.

При использовании этой опции возможен другой вариант вывода ошибки на вашей диаграмме:

- нет: две строки
- -stdrpt : одна строка
- -stdrpt:1 : многословный
- -stdrpt:2 : одна строка

[См. [выпуск № 155](#) и [QA-11805](#)]

Примеры с некорректным файлом file1.pu, где as написано aass:

```
@startuml
participant "Famous Bob" aass Bob
@enduml
```

Без всякого выбора

```
java -jar plantuml.jar file1.pu
```

Сообщение об ошибке выглядит следующим образом:

```
Error line 2 in file: file1.pu
Some diagram description contains errors
```

-опция stdrpt

```
java -jar plantuml.jar -stdrpt file1.pu
```

Сообщение об ошибке выглядит следующим образом:

```
file1.pu:2:error:Syntax Error?
```

-stdrpt:1 опция

```
java -jar plantuml.jar -stdrpt:1 file1.pu
```

Сообщение об ошибке выглядит следующим образом:

```
protocolVersion=1
status=ERROR
lineNumber=2
label=Syntax Error?
Error line 2 in file: file1.pu
Some diagram description contains errors
```

Опция -stdrpt:2 (аналогично -stdrpt)

```
java -jar plantuml.jar -stdrpt:2 file1.pu
```

Сообщение об ошибке выглядит следующим образом:

```
file1.pu:2:error:Syntax Error?
```

Стандартный ввод и вывод

Используя эту `-pipe` опцию, вы можете легко применять PlantUML в своих скриптах.

При выборе этого параметра описание диаграммы принимается через стандартный ввод, а файл PNG генерируется в стандартный вывод. Файл не записывается в локальную файловую систему.

Пример:

```
cat somefile.puml | java -jar plantuml.jar -pipe > somefile.png
```

Эта `-pipemap` опция позволяет генерировать данные карты в формате PNG (прямоугольники с гиперссылками) для использования в HTML, например:

```
cat somefile.puml | java -jar plantuml.jar -pipemap > somefile.map
```

Файл карты выглядит следующим образом:

```
<map id="plantuml_map" name="plantuml_map">
<area shape="rect" id="id1" href="http://plantuml.com" title="http://plantuml.com"
  alt="" coords="1,8,88,44"/>
</map>
```

Примечание: Также обратите внимание на `-pipedelimiter` реализацию `-pipeNoStderr` корректного мультиплексирования нескольких PNG-файлов в потоке (в случае, если файл `puml` содержит несколько диаграмм) и обработки ошибок.

  **Помощь**

Чтобы получить справочное сообщение, запустите:

```
java -jar plantuml.jar --help-more
```

В результате будет получен следующий результат:

plantuml - generate diagrams from plain text

Usage:

```
java -jar plantuml.jar [options] [file|dir]...
java -jar plantuml.jar [options] --gui
```

Description:

Process PlantUML sources from files, directories (optionally recursive), or stdin (-pipe).

Wildcards (for files/dirs):

- * any characters except '/' and '\'
 - ? exactly one character except '/' and '\'
 - ** any characters across directories (recursive)

Tip: quote patterns to avoid shell expansion (e.g., "***/*.puml").

General:

General:

```
--author ..... Show information about PlantUML authors
--check-graphviz ..... Check Graphviz installation
--dark-mode ..... Render diagrams in dark mode
--duration ..... Print total processing time
--gui ..... Launch the graphical user interface
-h, --help ..... Show help and usage information
```

последовательность прецедентов классов деятельность компонент состояний объект развертывание синхронизации

```
--http-server[:<port>] ..... Start internal HTTP server for rendering (default port : 8080)
--progress-bar ..... Show a textual progress bar
--splash-screen ..... Show splash screen with progress bar
-v, --verbose ..... Enable verbose logging
--version ..... Show PlantUML and Java version
```

Input & preprocessing:

```
--charset <name> ..... Use a specific input charset
--config <file> ..... Specify configuration file
-d, --define <VAR>=<value> ..... Define a preprocessing variable (equivalent to '!define <var> <value>')
--exclude <pattern> ..... Exclude input files matching the given pattern
-I, --include <file> ..... Include external file (as with '!include <file>')
-p, --pipe ..... Read source from stdin, write result to stdout
-P, --pragma <key>=<value> ..... Set pragma (equivalent to '!pragma <key> <value>')
--skinparam <key>=<value> .... Set skin parameter (equivalent to 'skinparam <key> <value>')
--theme <name> ..... Apply a theme
```

Execution control:

```
--check-before-run ..... Pre-check syntax of all inputs and stop faster on error
--check-syntax ..... Check diagram syntax without generating images
--graphviz-timeout <seconds> Set Graphviz processing timeout (in seconds)
--ignore-startuml-filename ... Ignore '@startuml <name>' and always derive output filenames from input files
--no-error-image ..... Do not generate error images for diagrams with syntax errors
--stop-on-error ..... Stop at the first syntax error
--threads <n|auto> ..... Use <n> threads for processing (auto = available processors)
```

Metadata & assets:

```
--decode-url <string> ..... Decode a PlantUML encoded URL back to its source
--disable-metadata ..... Do not include metadata in generated files
--dot-path <path-to-dot-exe> Specify the path to the Graphviz 'dot' executable
--encode-url ..... Generate an encoded PlantUML URL from a source file
--extract-source ..... Extract embedded PlantUML source from PNG or SVG metadata
--ftp-server ..... Start a local FTP server for diagram rendering (rarely used)
```

```
--list-keywords ..... Print the list of PlantUML language keywords
--skip-fresh ..... Skip PNG/SVG files that are already up-to-date (using metadata)
--sprite <4|8|16[z]> <file> .. Encode a sprite definition from an image file
```

Output control:

```
--output-dir <dir> ..... Generate output files in the specified directory
--overwrite ..... Allow overwriting of read-only output files
```

Output format (choose one):

```
-f, --format <name> ..... Set the output format for generated diagrams
                             (e.g. png, svg, pdf, eps, latex, txt, utxt, obfuscate, preproc...)
```

Available formats:

```
--eps ..... Generate images in EPS format
--html ..... Generate HTML files for class diagrams
--latex ..... Generate LaTeX/TikZ output
--latex-nopreamble ..... Generate LaTeX/TikZ output without preamble
--obfuscate ..... Replace text in diagrams with obfuscated strings to share diagrams safely
--pdf ..... Generate PDF images
--png ..... Generate PNG images (default)
--preproc ..... Generate the preprocessed source after applying !include, !define... (no rendering)
--scxml ..... Generate SCXML files for state diagrams
--svg ..... Generate SVG images
--txt ..... Generate ASCII art diagrams
--utxt ..... Generate ASCII art diagrams using Unicode characters
--vdx ..... Generate VDX files
--xmi ..... Generate XMI files for class diagrams
```

Statistics:

```
--disable-stats ..... Disable statistics collection (default behavior)
--enable-stats ..... Enable statistics collection
--export-stats ..... Export collected statistics to a text report and exit
--export-stats-html ..... Export collected statistics to an HTML report and exit
--html-stats ..... Output general statistics in HTML format
--loop-stats ..... Continuously print usage statistics during execution
--realtime-stats ..... Generate statistics in real time during processing
--xml-stats ..... Output general statistics in XML format
```

Examples:

```
# Process all .puml recursively
java -jar plantuml.jar "**/*.puml"

# Check syntax only (CI)
java -jar plantuml.jar --check-syntax src/diagrams

# Read from stdin and write to stdout (SVG)
cat diagram.puml | java -jar plantuml.jar --svg -pipe > out.svg

# Encode a sprite from an image
java -jar plantuml.jar --sprite 16z myicon.png

# Use a define
java -jar plantuml.jar -DAUTHOR=John diagram.puml

# Change output directory
java -jar plantuml.jar --format svg --output-dir out diagrams/
```

Exit codes:

See also:

`java -jar plantuml.jar --help-more`

Documentation: <https://plantuml.com>